

Senior Python Engineer

Technical Vetting Assignment

Instructions & Requirements

Take Home Assignment: Orders & Stock Core Flow (Python + PostgreSQL)

Context

Mr. D operates multiple e-commerce and logistics products. You're joining a backend team that builds transactional systems used by web and mobile apps, marketing, and warehouse tools. We need a small, reliable slice of that world: a minimal orders flow that updates stock and exposes clear interfaces other teams could build on.

Expectations in the role: move quickly in Python on Unix-like systems, work with PostgreSQL, design clean APIs, and shape components that can run and evolve independently. Temporary network glitches, duplicate submissions from clients, and service restarts happen in real life — the system should handle these everyday realities sensibly.

You can use AI-assisted coding tools you normally use in your workflow. Containerization is optional; the project must run locally on Linux or macOS. Minimal viable implementations are acceptable (for example: simple polling/outbox, no auth, a single process with a background worker). Simulating unavailability within a single process (e.g., pausing a worker) is acceptable.

Requirements

Build a minimal system in Python with PostgreSQL that accepts orders and updates stock, using your own API designs, data model, and component boundaries. Implement as one process or split into two if you think it's appropriate. Design components that can run and evolve independently.

Core technologies to use:

- Python
- PostgreSQL

Example data formats (illustrative; adapt as needed):

- Product
 - {sku: "BAN-001", name: "Bananas 1kg", price_cents: 199, stock: 50}
- Order
 - {order_ref: "web-100045", customer_id: "cust-42", items: [{sku: "BAN-001", qty: 2}, {sku: "APL-003", qty: 1}]}

Business constraints to guide your design (not implementation rules):

- An order might be submitted more than once with the same order_ref; only one should count.
- Orders should still be accepted during brief periods when the stock capability isn't immediately reachable; stock should catch up after recovery.
- Persist data in PostgreSQL. Don't rely solely on in-memory state.
- Demonstrate exactly two representative unhappy paths:
 1. duplicate submissions, and
 2. a brief interruption/outage simulation with catch-up afterward.

Task 1: Orders intake

Design and implement an Orders capability that:

- Accepts a new order and stores it in PostgreSQL.
- Prevents the same order from being counted twice if it is submitted more than once with the same order_ref.

- Calculates an order total from product prices at the time of the order.
- Exposes a way to create an order and to fetch order details/status.
- Includes a tiny script or command to seed a few products and submit a short burst of orders, intentionally including some duplicates of the same order_ref.

Keep this focused: no auth required; handle the two unhappy paths listed in the Requirements and keep any other edge cases out of scope.

Task 2: Stock updates, catch-up, and external surface OR report

Design and implement stock handling that:

- Manages products and stock levels in PostgreSQL.
- Reduces stock when an order is accepted by the Orders capability.
- Demonstrates how stock catches up after a brief interruption (a simple pause/resume of a worker or component is sufficient).
- Exposes a way to read current stock for a SKU.

Then choose exactly one of the following to complete this task:

- Option A: Integration surface
 - Provide a simple way for another system to consume “order accepted” information (your choice: feed, append-only log, webhook, polling endpoint, etc.).
 - Include a tiny consumer or script that shows how to receive/process these updates (printing to stdout is fine).
- Option B: Minimal report
 - Provide a lightweight endpoint (your design) that returns for a given day:
 - Total orders
 - Revenue
 - Units sold per SKU
 - Current stock per SKU
 - For this exercise, choosing this option satisfies the need for other teams to learn about accepted orders.

You decide how components communicate and how catch-up works. Keep the demonstration minimal but convincing.

Deliverables

1. A single GitHub repository containing:

- All the source code for the assignment
- A README.md with instructions to run locally.
- A SOLUTION.md describing the design and trade-offs you made.

2. Video Demo (5-10 minutes using Loom or similar):

- Demonstrate the working project by doing a voice over
- Walk through your code architecture
- Explain key design decisions and any trade-offs made
- Highlight interesting technical implementations

Time Estimate

This should take ± 4 hours to complete, but you can take as long as you need.